

TRAPPING RAIN WATER

Company: JP Morgan

Round: Online Assessment

Year: 2025

Difficulty: Difficult

Topic: Arrays, Two Pointers

Source: <https://www.lets-code.co.in/previousyearcodingquestion/jp-morgan-previous-year-coding-questions/>

Problem Description:

Imagine an elevation map made up of bars of different heights.

For example:

[0, 1, 0, 2, 1, 0, 1, 3, 2, 1, 2, 1]

After it rains, water can collect between the taller bars. Our task is to calculate how much water can be trapped between these bars. Each bar has a width of 1.

Important Idea

For any position, water can be stored only when there are taller bars on both sides. The amount of water at a position depends on the shorter of the two boundaries:

Water = $\min(\text{left boundary, right boundary}) - \text{current height}$

Input Format:

The input is an integer array height. Each value represents the height of a bar.

Output Format:

Return the total amount of water that can be trapped after raining.

Sample Input:

height = [0,1,0,2,1,0,1,3,2,1,2,1]

Sample Output:

Total Amount of Water = 6

Explanation:

Let's look at the idea behind the example.

[0, 1, 0, 2, 1, 0, 1, 3, 2, 1, 2, 1]

Water gets trapped in the spaces between taller bars.

For example, around this section:

2 | 1 | 0 | 1 | 3

the bars of height 2 and 3 form boundaries, so water can collect between them.

At every position, we ask:

How high can the water rise here?

The water level is decided by the shorter boundary.

So:

water level = $\min(\text{leftMax}, \text{rightMax})$

Then we subtract the height of the current bar:

water at current position = $\min(\text{leftMax}, \text{rightMax}) - \text{height}[i]$

Adding the water from all positions gives : 6

So the answer is 6 units of water.

Approach to Solve This Problem:

We can solve this efficiently using the Two Pointer approach.

Instead of calculating the maximum height on the left and right for every position separately, we use two pointers : left and right.

Initially:

left = 0

right = height.length - 1

We also keep:

leftMax

rightMax

These store the highest bars we have seen from the left and right.

Step 1: Compare the Two Sides

We compare:

`height[left]`
`height[right]`

If the left side is smaller, we process the left side.

Otherwise, we process the right side.

Why?

Because the smaller side determines the maximum possible water level.

Step 2: Update leftMax or rightMax

If:

`height[left] > leftMax`

we update:

`leftMax = height[left]`

Otherwise, the current bar is lower than leftMax, so it can hold water.

The same idea is used for the right side.

Step 3: Calculate Trapped Water

Suppose:

`leftMax = 2`
`height[left] = 1`

Then:

`water = 2 - 1 = 1`

So we add 1 to the total.

Step 4: Move the Pointer

After processing a position:

`left++`

or:

right--

We continue until:

left $\geq$ right

At that point, all possible positions have been processed.

Why Two Pointers Work

The important observation is:

The smaller boundary decides how much water can be trapped.

Suppose we know:

leftMax = 5

rightMax = 3

The water cannot rise above 3, because it would flow over the right boundary.

Therefore, the smaller side is safe to process.

This allows us to solve the problem without creating separate left and right arrays.

Java Code:

```
1  import java.util.*;
2
3  public class Main {
4
5      public static int trap(int[] height) {
6
7          if (height.length == 0) {
8              return 0;
9          }
10
11         int left = 0;
12         int right = height.length - 1;
13
14         int leftMax = 0;
15         int rightMax = 0;
16
17         int water = 0;
18     }
```

```
9 while (left < right) {
10
11     if (height[left] <= height[right]) {
12
13         // Update left maximum
14         if (height[left] >= leftMax) {
15             leftMax = height[left];
16         } else {
17             water += leftMax - height[left];
18         }
19
20         left++;
21
22     } else {
23
24         // Update right maximum
25         if (height[right] >= rightMax) {
26             rightMax = height[right];
27         } else {
28             water += rightMax - height[right];
29         }
30
31         right--;
32     }
33 }
34
35 return water;
36 }
```

```
Run | Debug
public static void main(String[] args) {

    Scanner sc = new Scanner(System.in);

    int n = sc.nextInt();

    int[] height = new int[n];

    for (int i = 0; i < n; i++) {
        height[i] = sc.nextInt();
    }

    System.out.println(trap(height));

    sc.close();
}
```

Solution:

The Two Pointer approach starts with one pointer at each end of the array. We keep track of the highest bar seen from both sides using leftMax and rightMax.

At every step, we process the side with the smaller current height because that side determines the water level. If the current bar is lower than its maximum boundary, water can be trapped there. We add that amount to water and move the pointer forward.

Understanding the Important Part of the Code:

1. Two Pointers

```
int left = 0;
```

```
int right = height.length - 1;
```

left starts from the beginning, while right starts from the end.

They move towards each other.

2. Maximum Heights

```
int leftMax = 0;
```

```
int rightMax = 0;
```

- leftMax → highest bar found from the left
- rightMax → highest bar found from the right

These help us know how much water a position can hold.

3. Choose the Smaller Side

```
if (height[left] <= height[right])
```

If the left bar is smaller, we process the left side.

Otherwise, we process the right side.

The reason is that the smaller boundary limits the water level.

4. Update leftMax

```
if (height[left] >= leftMax)
```

```
{
```

```
    leftMax = height[left];
```

```
}
```

If the current bar is taller than anything we've seen on the left, it becomes the new maximum.

5. Calculate Water

```
water += leftMax - height[left];
```

If the current bar is lower than leftMax, there is space for water.

For example:

```
leftMax = 3  
height[left] = 1
```

So:

```
water = 3 - 1 = 2
```

6. Move the Pointer

```
left++;
```

or:

```
right--;
```

Once we process a position, we move towards the other side.

Complexity Analysis:

Time Complexity: $O(n)$

Each position is visited at most once by the two pointers.

Time Complexity = $O(n)$

Space Complexity: $O(1)$

We only use a few variables such as left, right, leftMax, rightMax, and water. No extra array is required.

Space Complexity = $O(1)$